

Revisão — Unidade 1: Back-End e Front-End

Resumo dos Conceitos

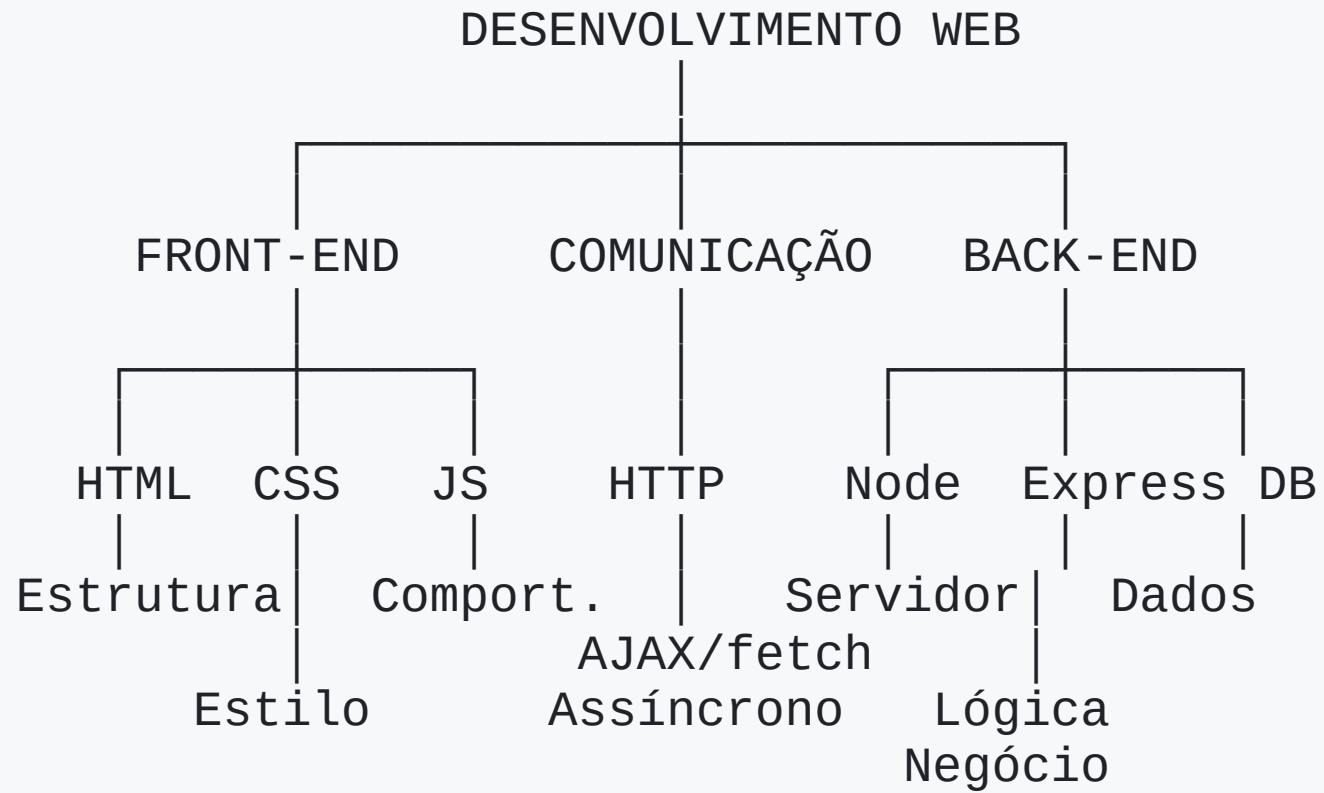
Aula 01 — Introdução ao Desenvolvimento Web

- Desenvolvimento de software é um **processo** que envolve análise, projeto, implementação, testes e manutenção;
- Aplicações web rodam no **navegador** e se comunicam com servidores via **HTTP**;
- A arquitetura **cliente-servidor** é o modelo fundamental da web;
- A web evoluiu de **estática** (1.0) para **dinâmica** (2.0) e **inteligente** (3.0);
- Existem diferentes paradigmas: **SPA**, **PWA**, **SSR**, **API-First**.

Aula 02 — O que é Front-End?

- Front-End é a **camada de apresentação** — o que o usuário vê e interage;
- Três pilares: **HTML** (estrutura), **CSS** (estilo), **JavaScript** (comportamento);

Mapa Conceitual



Conceitos Fundamentais

Conceito	Definição	Exemplo
Cliente	Navegador do usuário	Chrome, Firefox
Servidor	Máquina que processa requisições	Node.js, Apache
HTTP	Protocolo de comunicação	GET /api/usuarios
API	Contrato de comunicação entre sistemas	REST + JSON
DOM	Árvore do HTML em memória	document.querySelector()
Fetch	API do JS para requisições HTTP	fetch('/api/dados')
CRUD	Create, Read, Update, Delete	POST, GET, PUT, DELETE
JSON	Formato de troca de dados	{"nome": "Maria"}

Erros Comuns

Erro	Consequência	Correção
Usar <code>div</code> para tudo	Semântica ruim	Usar tags semânticas
Não validar dados no servidor	Dados inválidos no banco	Validar sempre no Back-End
Não tratar erros do fetch	Bugs silenciosos	Usar try/catch
Hardcodar credenciais	Falha de segurança	Usar variáveis de ambiente
Commitar <code>.env</code>	Credenciais expostas	Usar .gitignore
Confundir autenticação com autorização	Falha de segurança	Implementar ambos

Perguntas para Reflexão

1. **Por que o HTTP é stateless?** Quais são as vantagens e desvantagens dessa característica?
2. **Se o Front-End pode ser contornado,** por que ainda validamos dados nele?
3. **Qual é a diferença** entre uma API REST e uma API que apenas usa HTTP?
4. **Por que separar Front-End e Back-End** em vez de ter tudo em um único sistema?
5. **Se AJAX não existisse,** como seria a experiência do usuário em aplicações como Gmail e Facebook?
6. **Por que o código limpo importa** se o computador não se importa com nomes de variáveis?
7. **Qual é o papel do middleware** no ciclo de vida de uma requisição?

Checklist de Aprendizagem

Ao final da Unidade 1, o estudante deve ser capaz de:

- ☐ Explicar o que é desenvolvimento web e sua evolução;
- ☐ Descrever a arquitetura cliente-servidor;
- ☐ Identificar as três tecnologias do Front-End (HTML, CSS, JS);
- ☐ Explicar o que é o DOM e como o JavaScript o manipula;
- ☐ Descrever as responsabilidades do Back-End;
- ☐ Listar as principais tecnologias de Back-End;
- ☐ Explicar o protocolo HTTP e seus métodos;
- ☐ Diferenciar GET, POST, PUT e DELETE;
- ☐ Interpretar status codes HTTP;
- ☐ Explicar o que é uma API RESTful;

Conexão com a Unidade 2

Na Unidade 1, construímos aplicações **manualmente** — definindo rotas, middlewares e controllers "do zero". Isso foi fundamental para compreender como as coisas funcionam por dentro.

Na **Unidade 2 — Frameworks**, vamos estudar como **frameworks** como Express, React e outros **organizam e automatizam** grande parte desse trabalho. Você vai perceber que muitas das tarefas que fizemos manualmente — roteamento, parsing de JSON, servir arquivos estáticos — são resolvidas automaticamente por frameworks.

Pergunta de transição: Se já sabemos construir tudo manualmente, por que precisamos de frameworks?